**Instituto Tecnológico y de Estudios Superiores de Monterrey**

# Actividad 1: Predicción de Precios de Vehículos con XGBoost

**Una actividad presentada al**

Instituto Tecnológico y de Estudios Superiores de Monterrey

**Como Parte de la Unidad de Formación**

Implementación de Redes de Área Amplia y Servicios Distribuidos

Profesor

**Marcial Roberto Leyva Fernández**

Autor

**Víctor Alonzo Estévez Chávez**

Fecha

15 de abril del 2026

# Contents


**1 Configuración del Frontend en Next.js** **2**
1.1 Objetivo del Frontend . . . 2
1.2 Arquitectura General de la Solución . . . 2
1.2.1 Diagrama Entidad Relación tras la Transformación . . . 3
1.3 Estructura de Datos Interna en el Cliente . . . 3
1.4 Preprocesamiento Matemático de Variables Numéricas . . . 3
1.4.1 Funciones Matemáticas de Transformación para Características Particulares . . . 4
1.4.2 Observaciones Técnicas . . . 4
1.5 Tratamiento y Encoding de Variables Categóricas . . . 5
1.5.1 Realización de un Smooth Target Encoding . . . 5
1.5.2 Modo de Uso del Encoding para Variables Categóricas . . . 5
1.6 Orden de Variables y Compatibilidad con el Modelo . . . 5
1.7 Conexión del Frontend con AWS . . . 6
1.7.1 Contrato de Comunicación . . . 6
1.7.2 Manejo de Respuesta y Errores . . . 6
1.8 Flujo de Funcionamiento de Extremo a Extremo . . . 6
1.9 Configuración para Ejecución Local desde GitHub . . . 7
1.9.1 Requisitos . . . 7
1.9.2 Pasos para Ejecución en Entorno Local . . . 7
1.9.3 Verificación Recomendada en Localhost . . . 7

**2 Configuración del Entorno en Amazon Web Services** **8**
2.1 Requisitos Previos . . . 8
2.2 Estructura Mínima del Proyecto . . . 8
2.3 Instalación y Configuración Local de Docker . . . 8
2.3.1 Verificación de Instalación . . . 8
2.4 Configuración de Amazon Elastic Container Registry (ECR) . . . 9
2.4.1 Autenticación en Amazon ECR . . . 9
2.4.2 Configuración del Repositorio en Amazon ECR . . . 9
2.5 Construcción de la Imagen en Docker . . . 9
2.5.1 Importancia del Etiquetado . . . 9
2.6 Publicación de la Imagen en Amazon ECR . . . 10
2.6.1 Actualización del Código de la Función . . . 10
2.6.2 Corroborar la Versión Actual de la Imagen en ECR y Lambda . . . 10
2.7 Configuración de Memoria y Tiempo de Ejecución . . . 11
2.8 Capa de Ejecución y Manejo del Modelo . . . 11
2.8.1 Explicación General del Funcionamiento del Modelo en la Función Lambda . . . 11
2.9 Configuración del Permiso de Ejecución y CloudWatch . . . 11
2.9.1 Política Mínima Requerida . . . 11
2.9.2 Política Administrada Recomendada . . . 12
2.10 Creación y Configuración de API Gateway . . . 12
2.10.1 Creación de la API . . . 12
2.10.2 Habilitación de CORS . . . 12
2.11 Orden de las Variables de Entrada . . . 13
2.12 Resultado Final Esperado . . . 13

# 1 Configuración del Frontend en Next.js

Esta sección describe de forma detallada la arquitectura del frontend implementado en Next.js, la lógica matemática de preprocesamiento de datos, el tratamiento de variables categóricas, el flujo de inferencia hacia AWS y la configuración para ejecutar el proyecto en entorno local.

## 1.1 Objetivo del Frontend

El frontend tiene como finalidad el actuar como capa de preparación y validación de entrada antes de invocar al modelo de regresión `XGBoost` que se alojará en el servicio de Amazon Web Services en las secciones posteriores. Por lo tanto, los objetivos específicos para esta sección son descritos a continuación:

- Capturar atributos del vehículo mediante un formulario web.
- Transformar variables numéricas al rango esperado por el modelo.
- Convertir variables categóricas a una representación numérica compatible con la fase de entrenamiento.
- Mantener consistencia estricta en el orden y semántica de variables de entrada.
- Consumir un endpoint HTTP desplegado en Amazon API Gateway.
- Mostrar el precio estimado retornado por la función Lambda.

## 1.2 Arquitectura General de la Solución

A nivel funcional, el flujo completo del frontend puede resumirse en los siguientes pasos:

1. Renderizado del formulario con campos numéricos y categóricos.
2. Actualización del estado interno `formData` en cada cambio de campo.
3. Construcción de valores transformados mediante funciones matemáticas y tablas de mapeo.
4. Envío del estado completo al endpoint de API Gateway mediante solicitud `POST`.
5. Ejecución de inferencia en AWS Lambda sobre el modelo XGBoost.
6. Retorno de la predicción al frontend y despliegue del resultado al usuario.

### 1.2.1 Diagrama Entidad Relación tras la Transformación

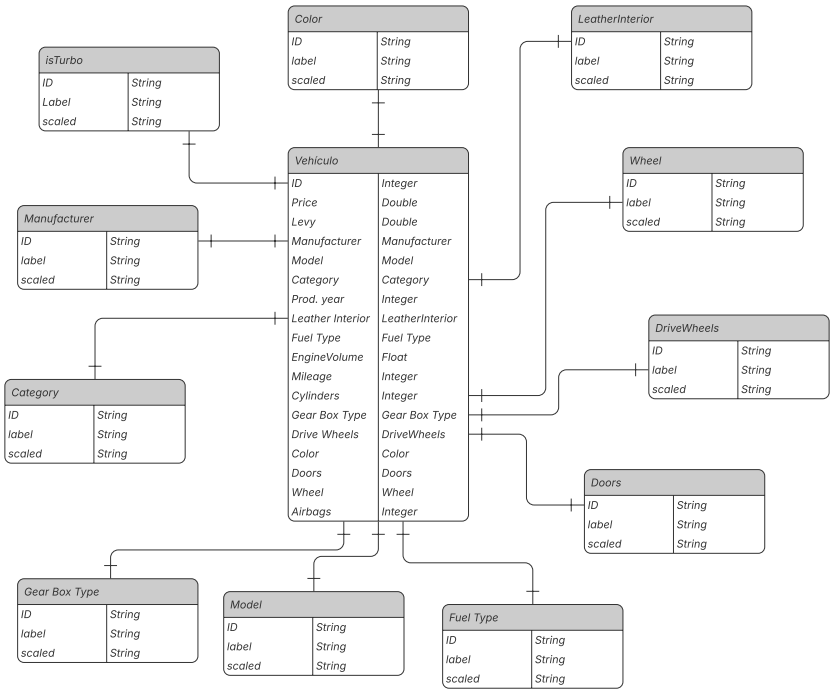


Figure 1: Diagrama de Entidad Relación.

## 1.3 Estructura de Datos Interna en el Cliente

El formulario que fue configurado en el frontend es controlado a través de una estructura de datos denominada `formData` en la que se almacena cada una de las variables que pueden ser ingresadas por el usuario mediante campos `input` y `select`, siendo algunos ejemplos: la categoría del vehículo, kilometraje, modelo, entre otras.

Por otro lado, cada variable de entrada se modela con dos componentes:

- `inputValue`: valor original ingresado o seleccionado por el usuario.
- `transformation`: valor numérico transformado que se usa para el entrenamiento del modelo. Este valor resulta tras aplicar las transformaciones matemáticas que derivan del preprocesamiento de datos.

De esta forma, el frontend conserva simultáneamente el valor capturado en la interfaz y su contraparte procesada para el modelo.

## 1.4 Preprocesamiento Matemático de Variables Numéricas

La transformación utilizada para variables numéricas corresponde a una normalización min-max. La forma general es:

$$x_{scaled} = \frac{x - x_{min}}{x_{max} - x_{min}}$$

Donde:

- $x$: valor original capturado.
- $x_{min}$: valor mínimo observado durante la preparación de datos.
- $x_{max}$: valor máximo observado durante la preparación de datos.

La aplicación de esta ecuación ocasiona que el rango de datos tratado se encuentre delimitado por [0,1].

### 1.4.1 Funciones Matemáticas de Transformación para Características Particulares

Las siguientes funciones matemáticas fueron obtenidas tras realizar un preprocesamiento de datos y son utilizadas para llenar ciertos campos dentro de la estructura de datos `formData`, particularmente su atributo `transformation`. Por otro lado, estas fórmulas pueden definirse como aplicaciones concretas de la normalización min-max.

$$\text{Levy}_{scaled} = \frac{x - 0}{11714 - 0} = \frac{x}{11714}$$

$$\text{ProdYear}_{scaled} = \frac{x - 1943}{2020 - 1943} = \frac{x - 1943}{77}$$

$$\text{EngineVolume}_{scaled} = \frac{x - 0.0}{20.0 - 0.0} = \frac{x}{20}$$

$$\text{Mileage}_{scaled} = \frac{x - 0}{2147483647 - 0} = \frac{x}{2147483647}$$

$$\text{Cylinders}_{scaled} = \frac{x - 1.0}{16.0 - 1.0} = \frac{x - 1}{15}$$

$$\text{Airbags}_{scaled} = \frac{x - 0}{16 - 0} = \frac{x}{16}$$

### 1.4.2 Observaciones Técnicas

- Estas transformaciones convierten magnitudes heterogéneas a una escala comparable.
- En la implementación actual no se aplica recorte explícito de rango, por lo que entradas fuera de intervalo producirán valores menores a 0 o mayores a 1.
- El valor transformado se serializa como texto para mantener compatibilidad con el contrato de la API.

## 1.5 Tratamiento y Encoding de Variables Categóricas

### 1.5.1 Realización de un Smooth Target Encoding

La técnica que fue implementada con el propósito de llevar a cabo una transformación de variables categóricas a valores numéricos, se denomina "Smooth Target Encoding" y se define como un procedimiento de ingeniería de características en el que se realiza un reemplazo de valores categóricos por la media de sus correspondientes variables objetivo, realizando a su vez una transformación de suavizado para lograr un balance entre el promedio de la categoría con el promedio global de la variable objetivo.

La ecuación utilizada fue la siguiente:

$$\text{Smoothed Value} = \frac{\text{Mean(Target} \mid \text{Category)} \times \text{Count(Category)} + \text{Mean(Target)} \times \text{Smoothing Parameter}}{\text{Count(Category)} + \text{Smoothing Parameter}}$$

Este procedimiento se llevó a cabo para cada una de las siguientes variables categóricas, tomando en cuenta un factor de suavizado de 10: Manufacturer, Model, Category, Fuel type, Gear box type, Drive wheels, Doors, Wheel, Color y Leather interior.

### 1.5.2 Modo de Uso del Encoding para Variables Categóricas

Las variables categóricas se procesan mediante un mapeo determinístico usando el archivo `Database.json`. Cada categoría contiene:

- `label`: texto mostrado al usuario.
- `scaled`: valor numérico precomputado usado por el modelo.

Este mecanismo puede entenderse como un encoding categórico por diccionario o tabla de búsqueda preentrenada. Es decir, la representación numérica de cada categoría no se calcula en tiempo real, sino que se reutiliza desde artefactos producidos durante entrenamiento/preprocesamiento mediante el procedimiento que es descrito en la sección 1.5

Por otro lado, es importante mencionar que el frontend realiza dos tipos particulares de codificación categórica debido a la propia naturaleza de los campos a través de los que opera. Particularmente, estas modalidades son las descritas a continuación:

1. Campos de tipo `select`: el valor de opción ya corresponde a `scaled`, por lo que la transformación es directa.
2. Campos `Manufacturer` y `Model`: se ingresa texto con sugerencias, luego se busca la mejor coincidencia en el catálogo para recuperar `scaled`.

Por último, cabe mencionar que uno de los aspectos más importantes para tener en cuenta está relacionado con la condición de que si `Manufacturer` o `Model` contienen un valor textual no coincidente con una etiqueta conocida, entonces no se obtendrá el valor mapeado esperado.

## 1.6 Orden de Variables y Compatibilidad con el Modelo

La consistencia en el orden de variables es crítica. El frontend debe preservar exactamente la secuencia usada en entrenamiento y en la lógica de inferencia de Lambda:

1. Manufacturer
2. Model
3. category
4. fuelType
5. gearBoxType
6. driveWheels
7. doors
8. wheel
9. color
10. leatherInterior
11. isTurbo
12. levy
13. prodYear
14. engineVolume
15. mileage
16. cylinders
17. airbags

Cualquier desalineación en orden de atributos puede provocar predicciones incorrectas aunque el servicio responda sin error.

## 1.7 Conexión del Frontend con AWS

El frontend realiza una solicitud HTTP `POST` hacia un endpoint de Amazon API Gateway desplegado en la región `us-east-2`. La API reenvía la carga útil a una función AWS Lambda que contiene al modelo XGBoost.

### 1.7.1 Contrato de Comunicación

- Método: `POST`
- Encabezado: `Content-Type: application/json`
- Cuerpo: objeto JSON derivado de `formData`
- Respuesta esperada: objeto con campo `result` o campo `error`

### 1.7.2 Manejo de Respuesta y Errores

El cliente implementa una validación por etapas:

- Verifica código HTTP exitoso.
- Intenta parsear JSON.
- Verifica ausencia de campo `error`.
- Convierte `result` a número y lo formatea con dos decimales.

Si alguna etapa falla, se reporta error al usuario.

## 1.8 Flujo de Funcionamiento de Extremo a Extremo

La secuencia operativa completa es la siguiente:

1. El usuario ingresa datos del vehículo.
2. El frontend actualiza estado y genera transformaciones numéricas.
3. Se envía la solicitud al endpoint de API Gateway.
4. API Gateway invoca Lambda.
5. Lambda reconstruye el vector de entrada en el orden requerido.
6. XGBoost infiere el precio estimado.
7. El resultado retorna al frontend y se muestra al usuario.

## 1.9 Configuración para Ejecución Local desde GitHub

### 1.9.1 Requisitos

- Node.js versión LTS (recomendado 20 o superior).
- npm instalado.
- Acceso a internet para crear y consumir la API en AWS.

### 1.9.2 Pasos para Ejecución en Entorno Local

1. Clonar el repositorio y acceder al proyecto con:

```
git clone https://github.com/V-Alonzo/CarPricePredictionWithAWS.
    git
```

2. Instalar dependencias:

```
npm install
```

3. Ejecución del proyecto:

```
npm run dev
```

### 1.9.3 Verificación Recomendada en Localhost

1. Completar formulario con valores válidos.
2. Ejecutar estimación.
3. Confirmar recepción de precio estimado.
4. Validar ausencia de errores CORS y errores HTTP.

Con esta configuración, el frontend en Next.js queda documentado como una capa de preparación de datos y orquestación de inferencia, estrechamente acoplada al contrato de entrada del modelo y al entorno de ejecución en AWS.

# 2 Configuración del Entorno en Amazon Web Services

Esta sección tiene el principal propósito de detallar de forma secuencial y estructurada, el procedimiento completo para desplegar la función AWS Lambda, estando empaquetada como imagen de contenedor (Docker), conectada a un repositorio de Amazon Elastic Container Registry (ECR) y expuesta a través de una Amazon API Gateway. Al finalizar el procedimiento descrito a lo largo de esta sección, se espera haber logrado la completa replicación de la solución planteada para su uso en un entorno de pruebas.

## 2.1 Requisitos Previos

Antes de iniciar el despliegue, se requiere lo siguiente:

- Una cuenta activa de AWS con permisos para trabajar con Lambda, ECR, API Gateway, IAM y CloudWatch.
- AWS CLI instalado y configurado con credenciales válidas.
- Docker instalado en el equipo local.
- Acceso al repositorio GitHub que contiene el código fuente de la función Lambda, el modelo serializado `model2.json` y el `Dockerfile`.
- Arquitectura compatible con la imagen objetivo `linux/amd64`, debido a que AWS Lambda ejecuta contenedores sobre esta plataforma en este escenario.

## 2.2 Estructura Mínima del Proyecto

Tras realizar la clonación del repositorio alojado en Github, debe tenerse acceso a los siguientes elementos dentro de la ruta "preprocessing/Python/AWS/Docker/project":

- `lambda_function.py`: archivo principal con la función `lambda_handler`.
- `model.json`: modelo entrenado y serializado.
- `Dockerfile`: definición de la imagen de contenedor.
- `requirements.txt`: dependencias de Python necesarias para el runtime.

## 2.3 Instalación y Configuración Local de Docker

Para construir la imagen que será enviada a Amazon ECR, Docker debe estar instalado y funcionando correctamente en el equipo local.

### 2.3.1 Verificación de Instalación

Se recomienda validar la instalación con los siguientes comandos:

```
docker --version
docker buildx version
```

Si el comando `buildx` no está disponible, debe habilitarse dado que permite construir imágenes para arquitecturas específicas como `linux/amd64`.

## 2.4 Configuración de Amazon Elastic Container Registry (ECR)

Se debe crear un repositorio de contenedores para almacenar la imagen de la Lambda, esto puede hacerse desde la consola de AWS o mediante AWS CLI siguiendo los pasos descritos en las secciones subsecuentes.

### 2.4.1 Autenticación en Amazon ECR

Se debe ingresar el siguiente comando para obtener las credenciales que serán necesarias para realizar las configuraciones correspondientes en el servicio de Amazon ECR.

```
aws ecr get-login-password --region us-east-2 | \
docker login --username AWS --password-stdin <ACCOUNT_ID>.dkr.ecr.us-east
    -2.amazonaws.com
```

La ubicación debe ser consistente y compartida para todos los comandos que así la requieran. Se estará haciendo uso de la región "us-east-2" en este caso sin ninguna razón crítica que la establezca como constante necesaria.

### 2.4.2 Configuración del Repositorio en Amazon ECR

Una vez que ha sido conseguida una autenticación exitosa, puede procederse a la creación del repositorio donde se almacenará la imagen correspondiente a la función Lambda, debe usarse el siguiente comando para tal propósito.

```
aws ecr create-repository \
  --repository-name xgb-lambda \
  --region us-east-2
```

El nombre del repositorio debe ser consistente con el flujo de despliegue. En este caso, el repositorio se denomina `xgb-lambda`.

## 2.5 Construcción de la Imagen en Docker

La imagen debe construirse explícitamente para la arquitectura `linux/amd64`. Por otro lado, se recomienda deshabilitar "provenance" y forzar una reconstrucción limpia de tal modo que se eviten inconsistencias derivadas del caché de construcción.

```
docker buildx build \
  --platform linux/amd64 \
  --provenance=false \
  --no-cache \
  -t <ACCOUNT_ID>.dkr.ecr.us-east-2.amazonaws.com/xgb-lambda:v<
      VERSION_NUMBER> \
   .
```

### 2.5.1 Importancia del Etiquetado

Se recomienda que cada cambio realizado cuente con un número de versión único y secuencial. De esta manera, se asegura una completa trazabilidad sobre cada uno de los cambios que son realizados en la imagen del contenedor, logrando consecuentemente un eficaz reconocimiento de aquello que está siendo ejecutado por los diferentes servicios de AWS en todo momento. Este valor puede ser reconocido mediante la etiqueta `<VERSION_NUMBER>` que aparece en los diferentes comandos.

## 2.6 Publicación de la Imagen en Amazon ECR

Una vez construida la imagen, debe enviarse al repositorio ECR mediante el siguiente comando:

```
docker push <ACCOUNT_ID>.dkr.ecr.us-east-2.amazonaws.com/xgb-lambda:v<
    VERSION_NUMBER>
```

En caso de que la imagen haya sido construida con una etiqueta local distinta, puede ser necesario asignar un nuevo nombre antes del `push`:

```
docker tag xgb-lambda:v<VERSION_NUMBER> <ACCOUNT_ID>.dkr.ecr.us-east-2.
    amazonaws.com/xgb-lambda:v<VERSION_NUMBER>
```

### 2.6.1 Actualización del Código de la Función

Es importante hacer del conocmiento que el comando inferior debe ejecutarse toda vez que se realizan cambios en la imagen del contenedor. En caso contrario, puede ocurrir que se tenga un despliegue adecuado en ECR, pero la ejecución de una versión desactualizada por Lambda.

```
aws lambda update-function-code \
  --function-name xgb-lambda-docker \
  --image-uri \verb|<ACCOUNT_ID>|.dkr.ecr.us-east-2.amazonaws.com/xgb-
      lambda:v<VERSION_NUMBER> \
  --region us-east-2
```

### 2.6.2 Corroborar la Versión Actual de la Imagen en ECR y Lambda

Para corroborar que Lambda posee un despliegue equivalente a la versión más reciente alojada en ECR, pueden ejecutarse los siguiente comandos tras haber efectuado el `push` con éxito.

El `push` debió haber arrojado una línea de texto en su parte más inferior donde puede identificarse la `key: "digest"`. Almacena este valor dado que será requerido más adelante. En caso de no haberlo obtenido por parte del `push`, entonces puede ingresarse el siguiente comando:

```
aws ecr describe-images \
--repository-name xgb-lambda \
--region us-east-2
```

El valor de interés es el que aparezca en el nivel más superior del JSON resultante, bajo la `key: "imageDigest"`.

Tras haber obtenido adecuadamente uno de los valores de interés anteriormente descritos, puede procederse a ejecutar el siguiente comando:

```
aws lambda get-function \
--function-name xgb-lambda-docker \
--region us-east-2
```

Una vez que haya sido localizada la `key: CodeSha256` en el JSON resultante del comando, entonces puede compararse contra cualquiera de los dos valores mencionados. En caso de ser iguales, puede concluirse que tanto ECR como Lambda se encuentran ejecutando la misma versión de la imagen. En caso contrario, puede ser necesaria la ejecución del comando `update-function-code` que fue descrito en la sección 2.6.1

## 2.7 Configuración de Memoria y Tiempo de Ejecución

En el caso particular de modelos de XGBoost, la memoria asignada originalmente puede resultar insuficiente. Durante las pruebas se observó un consumo cercano al límite con `128 MB`, lo que provocaba timeouts. Por ello, se recomienda elevar la configuración de la función mediante el siguiente comando:

```
aws lambda update-function-configuration \
  --function-name xgb-lambda-docker \
  --memory-size 1024 \
  --timeout 15 \
  --region us-east-2
```

Este ajuste mejora el rendimiento, incrementa la memoria disponible y reduce la probabilidad de fallos por agotamiento de recursos.

## 2.8 Capa de Ejecución y Manejo del Modelo

La función Lambda carga el modelo `model2.json` y ejecuta la inferencia sobre los valores recibidos desde el frontend. Para reducir la latencia, el modelo debe cargarse una sola vez por contenedor y no en cada solicitud. De esta manera, el modelo puede ser reutilizado en invocaciones posteriores mientras el contenedor permanezca activo. Esta es la principal razón por la que su declaración e inicialización se encuentran fuera de la función `lambda_handler` declarada en el archivo fuente de la función Lambda: `lambda_function.py`.

### 2.8.1 Explicación General del Funcionamiento del Modelo en la Función Lambda

La lógica del modelo se estructura de la siguiente manera conceptual:

- Cargar el modelo fuera del `handler` principal.
- Validar la existencia del cuerpo de la solicitud.
- Convertir las transformaciones creadas en el frontend a valores numéricos `float` dado que la estructura de datos que se recibe mediante el método HTTP `post` los contiene en formato `string`.
- Mantener el orden de variables exactamente igual al usado durante el entrenamiento.
- Devolver la predicción como un valor JSON serializable.

## 2.9 Configuración del Permiso de Ejecución y CloudWatch

De tal forma que Lambda pueda escribir eventos en CloudWatch Logs, el rol de ejecución debe incluir permisos de logging. Sin esta configuración, la función puede ejecutarse, pero no generará registros útiles para depuración.

### 2.9.1 Política Mínima Requerida

El rol de ejecución asociado a la función debe incluir los siguientes permisos:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
```

```
            "logs:CreateLogGroup",
            "logs:CreateLogStream",
            "logs:PutLogEvents"
        ],
        "Resource": "*"
    }
  ]
}
```

### 2.9.2 Política Administrada Recomendada

Como alternativa, puede adjuntarse la política administrada de AWS:

- `AWSLambdaBasicExecutionRole`

De este modo, Lambda podrá registrar errores, advertencias y trazas de ejecución en CloudWatch.

## 2.10 Creación y Configuración de API Gateway

Para exponer la función Lambda mediante HTTP, se utiliza Amazon API Gateway.

### 2.10.1 Creación de la API

Los pasos generales son:

1. Iniciar sesión en la página de internet que permite el acceso a la consola de AWS. Las credenciales deben ser las mismas que las que han sido utilizadas durante todo el procedimiento.

2. Dirigirse al servicio "Lambda" para posteriormente seleccionar la función lambda recientemente creada bajo el nombre `xgb-lambda-docker`

3. Seleccionar la opción que permite agregar un nuevo desencadenador. Establecer el origen del desencadenador al valor `API Gateway`.

4. Seleccionar la opción que indica la creación de una nueva API, siendo su tipo: `REST API`

5. Establecer `open` para la configuración de seguridad.

6. Agregar la nueva Gateway API mediante el botón con la etiqueta `Agregar`.

### 2.10.2 Habilitación de CORS

Debido a que el frontend se ejecuta en un origen distinto al de la API como lo puede ser `localhost:3000` en ambientes de desarrollo, es necesario habilitar CORS.

**2.10.2.1 Encabezados en la Respuesta de Lambda** La función debe devolver encabezados CORS tanto en respuestas exitosas como en respuestas de error:

```
"headers": {
    "Access-Control-Allow-Origin": "*",
    "Access-Control-Allow-Headers": "*",
    "Access-Control-Allow-Methods": "OPTIONS,POST,GET"
}
```

Esta es la razón por la que el handler de la función Lambda definido en `lambda_function.py` contiene esto como parte del valor que devuelve.

#### 2.10.2.2 Configuración para CORS en la API Gateway

1. Iniciar sesión en la página de internet que permite el acceso a la consola de AWS. Las credenciales deben ser las mismas que las que han sido utilizadas durante todo el procedimiento.

2. Dirigirse al servicio "API Gateway" para posteriormente seleccionar la API recientemente creada bajo el nombre `xgb-lambda-docker-API`.

3. Seleccionar la opción `Habilitar CORS`.

4. Marcar las opciones `Default 4XX` y `Default 5XX`.

5. Guardar la configuración.

Después de habilitar CORS en API Gateway, es obligatorio volver a desplegar la API mediante el botón `Implementar API`. En caso contrario, los cambios no se aplicarán al endpoint publicado.

## 2.11 Orden de las Variables de Entrada

La predicción depende de que las variables se envíen en el mismo orden utilizado durante el entrenamiento del modelo. Por ello, el arreglo `keysParamsOrder` debe mantenerse estable y alineado con el preprocesamiento original.

```
keysParamsOrder = [
    "Manufacturer", "Model", "category", "fuelType", "gearBoxType",
    "driveWheels", "doors", "wheel", "color", "leatherInterior",
    "isTurbo", "levy", "prodYear", "engineVolume", "mileage",
    "cylinders", "airbags"
]
```

Cualquier cambio en este orden puede producir predicciones incorrectas o errores de compatibilidad entre atributos y modelo.

## 2.12 Resultado Final Esperado

Una vez completados los pasos anteriores, el flujo de ejecución debe operar de la siguiente forma:

1. El usuario captura los atributos del vehículo en el frontend.

2. El frontend transforma los datos y los envía mediante una solicitud `POST`.

3. API Gateway recibe la solicitud y la redirige a Lambda.

4. Lambda valida el cuerpo, transforma las variables y carga el modelo.

5. XGBoost genera el precio estimado.

6. La respuesta se devuelve al frontend en formato JSON.